

Classification Assignment

Problem Statement or Requirement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the Chronic Kidney Disease (CKD) based on the several parameters. The Client has provided the dataset of the same.

Ans:

1. Client wants to predict the chronic kidney disease based on the several parameter. According to the dataset provided by user, the 3-stage problem identification are:

- Machine learning
- Supervised learning
- Classification

2. Dataset's has 399 rows and 25 columns.

3. The dataset was split into two set as indep and dep.

4. And used standard scaler preprocessor to make the inputs into the defined range.

5. For model creation, we going to use use various classification algorithms and chose the best model.

Algorithm-1-Grid Logistic Regression:

```
[13]: from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}:".format(grid.best_params_),f1_macro)

The f1_macro value for best parameter {'penalty': 'l2', 'solver': 'sag'}: 0.9924946382275899
```

```
[14]: print("The confusion Matrix:\n",cm)

The confusion Matrix:
[[51  0]
 [ 1 81]]
```

```
[15]: print("The report:\n",clf_report)

The report:
      precision    recall  f1-score   support

      0       0.98      1.00      0.99         51
      1       1.00      0.99      0.99         82

 accuracy          0.99         133
 macro avg          0.99      0.99         133
 weighted avg          0.99      0.99         133
```

```
[16]: from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(X_test)[:,:1])
```

```
[16]: np.float64(1.0)
```

Algorithm-2-Grid KNN:

```
[13]: from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,averages='weighted')
print("The f1_macro value for best parameter {}".format(grid.best_params_),f1_macro)

The f1_macro value for best parameter {'metric': 'minkowski', 'n_neighbors': 7, 'p': 2}: 0.940494593126172

[14]: print("The confusion Matrix:\n",cm)

The confusion Matrix:
[[51  0]
 [ 8 74]]

[15]: print("The report:\n",clf_report)

The report:
      precision    recall  f1-score   support

    0       0.86       1.00       0.93        51
    1       1.00       0.90       0.95        82

 accuracy          0.94        133
 macro avg       0.93       0.95       0.94        133
 weighted avg    0.95       0.94       0.94        133

[16]: from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(X_test)[:,:1])

[16]: np.float64(0.9996413199426112)
```

Algorithm-3-Grid Naive’s Bayes:

 jupyter

Classification Assignment-Grid Naive's Bayes Last Checkpoint: 19 days ago



File Edit View Run Kernel Settings Help

JupyterLab Python 3 (ipykernel)

```
f1_macro=f1_score(y_test,grid_predictions,averages='weighted')
print("The f1_macro value for best parameter {}".format(grid.best_params_),f1_macro)

The f1_macro value for best parameter {'alpha': 0.01}: 0.9775556904684072

[11]: print("The confusion Matrix:\n",cm)

The confusion Matrix:
[[51  0]
 [ 3 79]]

[12]: print("The report:\n",clf_report)

The report:
      precision    recall  f1-score   support

    0       0.94       1.00       0.97        51
    1       1.00       0.96       0.98        82

 accuracy          0.98        133
 macro avg       0.97       0.98       0.98        133
 weighted avg    0.98       0.98       0.98        133

[13]: from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(X_test)[:,:1])

[13]: np.float64(0.9966523194643712)
```

Algorithm-4-Grid SVM:

```
Jupyter Classification Assignment-Grid SVM Last Checkpoint: 19 days ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)

[13]: from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}".format(grid.best_params_),f1_macro)

The f1_macro value for best parameter {'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}: 0.9924946382275899

[14]: print("The confusion Matrix:\n",cm)

The confusion Matrix:
[[51  0]
 [ 1 81]]

[15]: print("The report:\n",clf_report)

The report:
      precision    recall  f1-score   support

     0       0.98       1.00       0.99         51
     1       1.00       0.99       0.99         82

 accuracy          0.99          0.99          0.99         133
 macro avg          0.99          0.99          0.99         133
 weighted avg          0.99          0.99          0.99         133

[16]: from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(X_test)[:,:1])

[16]: np.float64(1.0)
```

Algorithm-5-Grid RF:

```
Jupyter Classification Assignment-Grid RF Last Checkpoint: 19 days ago
File Edit View Run Kernel Settings Help
JupyterLab Python

[16]: from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}".format(grid.best_params_),f1_macro)

The f1_macro value for best parameter {'criterion': 'entropy', 'max_features': 'sqrt', 'n_estimators': 100}: 0.9849624060150376

[17]: print("The confusion Matrix:\n",cm)

The confusion Matrix:
[[50  1]
 [ 1 81]]

[18]: print("The report:\n",clf_report)

The report:
      precision    recall  f1-score   support

     0       0.98       0.98       0.98         51
     1       0.99       0.99       0.99         82

 accuracy          0.98          0.98          0.98         133
 macro avg          0.98          0.98          0.98         133
 weighted avg          0.98          0.98          0.98         133

[19]: from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(X_test)[:,:1])

[19]: np.float64(0.9997608799617408)
```

After creating 6 models, we could see the roc_auc_score of grid Logistic Regression and grid SVM has 1. So, I'm finalizing the both are the best model